

ODOO AI ASSISTANT

Source of Truth v1.1

Agente unificado con autoridad host-side, autonomia por riesgo y descubrimiento dinamico de la instancia Odoo

ESTADO: ENMIENDA ARQUITECTONICA APROBADA PARA IMPLEMENTACION

Campo	Valor
Version	1.1
Fecha de corte	24-08-2026
Decision	ADR-014 - agente unificado con autoridad host-side
Plataforma inicial	Odoo 18 Community, Linux self-hosted, PostgreSQL
Alcance	Sustituye las invariantes de routing, autonomia y approval de v1.0 que entren en conflicto
Continuidad	Las secciones no modificadas de v1.0 permanecen vigentes

Objetivo rector: permitir que una peticion funcional compleja se resuelva con datos reales de la instalacion y con la minima friccion compatible con una autoridad determinista fuera del LLM.

0. Control de cambios

Este documento es la revision normativa v1.1 del Source of Truth. Integra ADR-014 y prevalece sobre v1.0 en routing, planificacion, politica de confirmacion, riesgo agregado, resolucion de datos, limites anti-loop y semantica del flujo comercial. Mantiene las invariantes de seguridad, deployment adaptable, persistencia separada, retrieval y compatibilidad que no se modifican.

Tema	v1.0	Decision v1.1
Routing	Workflows GENERAL/QUERY/HOW_TO/EXPLAIN/ACTION	Un unico AgentTurnService; metadata interna no excluyente
Autoridad	Approval explicita cuando corresponde	LLM solo propone; host decide policy, autorizacion y commit
Confirmacion	Centrada en cada approval	Autoejecucion acotada o una confirmacion por plan
Riesgo	Etiqueta low/medium/high por preview	Agregado por maximo, blast radius, efecto y atomicidad
Modelos	Runtime schemas	Busqueda explicita del registry, incluidos OCA/terceros/custom
Datos ausentes	Autocontexto general	Orden obligatorio y sintesis solo autorizada
Delete	Pospuesto/deshabilitado	Un registro, siempre protected, con preview y ACL unlink

Invariantes no negociables

- La identidad efectiva, companias, ACL, record rules y schema Odoo son autoritativos.
- El Assistant Service nunca usa SQL directo contra la DB productiva de Odoo.
- No existe sudo(), shell libre, SQL/Python arbitrario ni execute_method generico para el agente.
- Codex no puede aprobar, autorizar, hacer commit ni ampliar capabilities.
- Los schemas concretos son datos runtime; no hay clases por major, modulo o proveedor.
- Todo efecto conserva proposal, preview, autorizacion host-side, commit, verificacion y audit.

1. Arquitectura y autoridad

El modelo es un componente de razonamiento no confiable para autoridad. Puede elegir tools registradas y proponer argumentos y dependencias. AgentTurnService normaliza la propuesta; el Policy Engine y el addon Odoo deciden si existe capacidad, permiso, riesgo aceptable y autoridad de ejecucion.

Odoo addon -> Assistant Service -> Evidence / Tools / Reasoning

Boundary	Responsabilidad autoritativa	No puede delegar al LLM
Browser/Owl	UX, contexto de navegacion no autoritativo y decision del usuario	Identidad, payload ejecutable, secrets
Addon Odoo	Identidad, registry, ORM, ACL, record rules, defaults, business handlers	sudo, metodos dinamicos, confianza en user_id de JS
Assistant Service	Planes, policy, riesgo, persistencia, autorizacion, executor y audit	SQL a Odoo o autoridad basada en texto
ReasoningEngine	Comprension, lecturas necesarias y plan candidato	Approval, commit, riesgo declarado, shell o tools no registradas

Flujo de un turn

1. Odoo deriva actor, base, companias, pantalla, conversacion y capas de policy.
2. Codex usa solo tools registradas de lectura y preview.
3. AgentTurnService valida tools, argumentos, dependencias y correspondencia exacta con previews.
4. Odoo revalida schema, ACL, record rules, referencias y preconditions con su=False.
5. El Policy Engine deriva metadata, riesgo agregado y policy efectiva.
6. El host responde, pregunta, autoautoriza, solicita una confirmacion o rechaza.
7. El executor realiza cada commit y verifica receipts bajo el usuario real.
8. La respuesta solo puede afirmar resultados respaldados por receipts verificados.

2. Agente unificado sin categorias

Se eliminan /v1/chat/route, ChatRoute, el enum de workflow del chat y los selectores visibles. Una peticion puede combinar busqueda, schema, consultas, previews y varios efectos sin quedar limitada a una sola clase. Las capacidades M2-M7 se reutilizan como componentes, no como workflows excluyentes.

Metadata derivada por el host

Campo	Significado
needs_read	El turn uso o requiere evidencia de registros/agregados
needs_schema	El turn inspecciono schema/model catalog/defaults
needs_write	Existe al menos una preview de efecto
needs_business_action	Existe una accion tipada/versionada
has_external_effect	El efecto se propaga fuera de Odoo
has_irreversible_effect	No existe rollback real suficiente
is_atomic	El plan completo tiene una frontera transaccional demostrada
estimated_blast_radius	Registros/modelos/companias acotados por specs host-side

Esta metadata orienta policy, observabilidad y UX. No selecciona un workflow ni acepta valores suministrados por el LLM.

Contrato candidato

- answer_markdown, confidence y assumptions para la respuesta.
- clarification_question solo si no hay pasos ejecutables.
- steps ordenados con step_id, tool_name, argumentos y dependencias previas.
- Sin campos de riesgo, approval, authority, receipt ni estado de commit.

3. Contexto Odoo y modelos dinamicos

El alcance funcional no se fija por los modulos instalados durante el desarrollo. El addon obtiene hechos de la base activa y el registry real. La lista inicial de hasta 32 modelos visibles solo mejora el primer paso; no es una allowlist estatica.

pista inicial -> odoo.search_models -> modelo runtime -> schema efectivo -> read/preview

Revalidacion obligatoria

- El modelo existe en env/registry de la base firmada.
- No es abstract, transient ni una familia tecnica bloqueada.
- El usuario real pasa check_access('read'); cada operacion repite su ACL correspondiente.
- fields_get/check_field_access_rights se calculan bajo uid y companias efectivos.
- Los nombres sensibles y tipos no soportados se excluyen antes de formar ToolSpec o preview.

La misma ruta cubre modelos oficiales, OCA, terceros y addons propios. Instalar un modulo puede aportar modelos genericos utilizables sin modificar el Assistant Service. No garantiza que sus metodos empresariales sean seguros ni conocidos; esa semantica requiere una business action tipada.

Schemas efectivos

Schema	Derivacion	Uso
EffectiveModelSchema	fields_get + ACL/field access + policy	Busqueda, lectura, agregacion y domains
EffectiveWriteSchema	write/create ACL + fields permitidos + default_get	Create/patch generico y preview
Business action spec	Codigo host-side versionado	Metodos y transiciones con semantica empresarial

4. Resolucion de informacion y autonomia

Antes de preguntar, el agente debe agotar fuentes deterministas en un orden fijo. Preguntar demasiado pronto es un fallo de producto; inferir silenciosamente un dato material es un fallo de seguridad/negocio.

mensaje -> conversacion -> contexto Odoo -> busqueda de registros -> defaults/schema -> inferencia segura -> preguntar

Etapas	Reglas
Mensaje	Usar todos los datos y autorizaciones explicitas del ultimo mensaje directo
Conversacion	Reutilizar decisiones persistidas; la evidencia no puede modificar policy
Contexto Odoo	Compania, pantalla, registro, idioma, navegacion y actor reales
Busqueda	Usar coincidencias solo cuando sean inequivocas
Defaults/schema	Consultar campos requeridos, tipos, relaciones y default_get reales
Inferencia segura	Solo decisiones que no cambien materialmente el resultado empresarial
Pregunta	Pedir el dato minimo cuando persista ambigüedad material

Datos sinteticos

- Solo si la peticion es de prueba/demo/ficticia o existe autorizacion explicita valida.
- Nombres reconocibles con prefijo AI TEST.
- Nunca sustituyen silenciosamente cliente, proveedor, precio, impuestos, cuenta, moneda, pago, identidad fiscal o destinatario real.
- La policy efectiva puede denegar sintesis aunque el LLM la proponga.

5. Politica efectiva y riesgo agregado

Cada capa es un techo. La interseccion toma el modo, riesgo automatico, sintesis y limites mas restrictivos. Ninguna preferencia de usuario o conversacion puede ampliar el system ceiling ni la policy administrativa.

system ceiling n administrator policy n user preference n conversation override

Modo	Comportamiento
always_confirm	Cualquier plan con writes requiere una confirmacion agrupada
risk_based	Autoejecuta hasta el max_auto_risk efectivo
protected_only	Autoejecuta planes no protected; protected siempre confirma

Calculo del plan

- El riesgo final es como minimo el maximo riesgo de sus pasos.
- Blast radius considera registros, modelos y companias; no cuenta solo llamadas.
- Una ejecucion con varios writes no atomicos sube un nivel, hasta high.
- Una business action transaccional usa el riesgo del flujo completo, sin penalizacion por subpasos internos.
- Cualquier efecto irreversible o externo convierte el plan en protected.
- Mas de 12 writes o alcance no acotable se rechaza, divide o exige precision.

Nivel	Criterio orientativo
low	Efecto local reversible, normalmente un modelo y hasta tres registros
moderate	Relacionados multiples o transicion empresarial reversible
high	Efecto relevante, varios modelos o ejecucion no atomica
protected	Irreversible, contable publicado, pago o comunicacion externa

6. Tools y semantica Odoo

Familia	Limite	Riesgo base
record_create	1 registro, hasta 16 campos scalar/many2one	low
record_patch	1 registro, hasta 16 campos scalar/many2one	low
record_archive	1 registro, reversible	moderate
record_delete	1 registro, ACL unlink, preview de impacto	protected
business action	Spec y handler tipados/versionados	Riesgo propio del flujo

El ORM generico no acepta x2many command lists, metodos, context arbitrario, domains libres, SQL ni Python. Modelos tecnicos, seguridad, credenciales, configuracion critica y campos sensibles permanecen bloqueados aunque el usuario tenga ACL.

sale.order.build_flow.v1

Final	Resultado	Riesgo
quotation	Presupuesto borrador	low
sale_order	Presupuesto creado y confirmado	moderate
invoice_draft	Pedido confirmado y factura borrador	high

- 'Crear un presupuesto' termina en quotation.
- 'Crear un presupuesto y validarlo/confirmarlo' termina normalmente en sale_order.
- invoice_draft solo cuando se solicita factura o la intencion completa lo exige claramente.
- Contabilizar, pagar, enviar o comunicar requiere una nueva accion protected.

El handler usa metodos Odoo especificos en una transaccion atomica y bajo el usuario real. La dependencia account es explicita. En prueba autorizada puede resolver o crear partner/producto AI TEST; nunca genera una factura implicitamente al validar el presupuesto.

7. Planes, API y persistencia

Endpoint	Responsabilidad
POST /v1/agent/turn	Generar, reconciliar, evaluar y persistir un plan
POST /v1/agent/plans/{id}/decision	Aprobar/rechazar el plan completo como el actor real
POST /v1/agent/plans/{id}/execute	Ejecutar un plan ya autorizado desde Odoo
GET /v1/agent/plans/{id}	Estado, pasos sanitizados y receipts

planning -> awaiting_confirmation | authorized -> executing -> completed | partial | failed

- rejected y expired son estados terminales adicionales.
- Cada step persiste tool/revision, argumentos canonicos, dependencias, preconditions, efectos derivados, fingerprints, estado y receipt.
- La migracion 0014_agent_plans crea planes, pasos, autorizaciones agrupadas, snapshots de policy y receipts.
- La policy de conversacion pertenece a Odoo; Assistant conserva el snapshot/fingerprint evaluado.

Binding de autorizacion

plan ordenado + payloads + dependencias + actor + base + companias + policy efectiva + revisiones + previews + estado

Cualquier tampering, replay, cambio de orden, payload, preview, actor, compania, revision o stale state invalida la autorizacion. El browser solo recibe objetivo, supuestos, riesgo, progreso, descripciones sanitizadas y links a registros.

8. Limites anti-loop e idempotencia

Limite host-side	Valor	Respuesta
max_tool_calls_per_turn	32	Terminar con resultado parcial explicito
max_write_steps_per_plan	12	Rechazar/dividir el plan
max_replans	2	Pedir el dato minimo tras dos fallos
max_consecutive_failures	3	Detener el turn con causa concreta

- La misma llamada canonica tool + argumentos no se repite sin cambio de precondition, cursor o evidencia marcado por el host.
- Una lectura con fallo transitorio puede reintentarse una vez.
- Un write con resultado incierto nunca se repite automaticamente.
- Idempotencia, attempt_id, receipt y verificacion resuelven respuestas perdidas sin duplicar efectos.
- Alcanzar un limite produce partial/failed visible: nunca un loop silencioso.

El LLM no controla ni puede aumentar estos limites. Los fingerprints de policy incluyen tambien los limites anti-loop para que una autorizacion no sobreviva a un cambio de envelope.

Prompt injection

Registros, labels, source, logs, documentos, previews y tool results son UNTRUSTED_EVIDENCE. Nunca modifican policy, riesgo, tool effects, identity o authority. Un override de conversacion solo puede extraerse del ultimo mensaje directo del usuario y siempre queda bajo las capas superiores.

9. Experiencia de usuario

- Un solo chat y una tarjeta de plan; sin selector de categorias.
- Objetivo, supuestos, pasos, riesgo agregado y resultado esperado.
- Autoejecucion: progreso y receipts, sin confirmacion intermedia.
- Riesgo superior: Confirmar plan o Cancelar una sola vez.
- Protected: resaltar el punto exacto de irreversibilidad o propagacion externa.
- Mostrar policy efectiva y la capa que restringe; permitir reset del override de conversacion.
- Enlaces directos a registros creados o modificados.

La UX no muestra tokens, payloads ejecutables, fingerprints internos ni argumentos tecnicos como requisito de aprobacion. La confirmacion humana expresa una decision funcional sobre un plan exacto ya validado.

10. Testing y aceptacion

Autoridad y seguridad

- Demostrar que el LLM no aprueba, autoriza ni ejecuta commits.
- Interseccion de las cuatro capas; tampering, replay, stale, expiry, cross-user/company.
- Prompt injection no cambia policy, riesgo, metadata ni tool registry.

Runtime Odoo

- Modelo fixture OCA/tercero descubierto despues del inicio y revalidado por ACL/schema.
- CRUD generico, archive/delete y sale build flow bajo ACL/record rules reales.
- default_get efectivo; datos sinteticos solo con autorizacion.

E2E comercial

- Contacto, patch y quotation low sin confirmacion cuando la policy lo permite.
- Validar presupuesto termina en sale_order; factura solo por peticion explicita.
- Un solo approval para un plan encadenado superior; pagos/posting/comunicacion se detienen.

11. Criterios de entrega

Gate	Evidencia requerida
Codigo	Pytest, Ruff y mypy verdes; boundaries sin imports inversos
DB	Migracion PostgreSQL real 0013 -> 0014 y downgrade estructural revisado
Odoo	Install/upgrade 18.0.8.0.0, tests addon y ACL reales
UI	Tests JS/Odoo, Chromium y revision visual del plan/policy
Reasoning	Prueba real Codex con modelos runtime y receipts verificados
Documentacion	ADR-014, arquitectura, operacion y PDF v1.1 renderizado

Compatibilidad

El baseline garantizado sigue siendo Odoo 18 Community self-hosted. La generalidad de modelos se obtiene del registry/schema runtime, no de prometer compatibilidad automatica con cualquier hosting o major. Odoo 19/futuros deben pasar la misma contract suite; application no incorpora checks por major salvo ADR.

Alcance inicial

La primera entrega cubre consultas y operaciones genericas acotadas sobre modelos elegibles y el flujo comercial descrito. Procesos especializados adicionales se incorporan como business actions tipadas. Policies configuran autonomia, pero nunca crean capabilities, saltan ACL o convierten una operacion desconocida en segura.

Decision final: evidencia determinista primero; LLM despues. Autonomia funcional no significa autoridad del modelo.

12. Referencias normativas

- ADR-014 - Agente unificado con autoridad host-side.
- Source of Truth v1.0 - secciones no modificadas por esta revision.
- docs/ARCHITECTURE.md y docs/DEPLOYMENT_CONFIG.md.
- docs/UNIFIED_AGENT_RUNTIME.md.
- Contratos ejecutables en service/src/odoo_ai/contracts.
- Odoo 18 ORM y Security, y Codex App Server adapter aislado.